



TWITCH ANALYTICS II

Las Grecas

ÍNDICE

Introducción	2
Objetivos	3
Consideraciones (probar implementación y dónde se encuentra alojada)	4
Detalles sobre implementación	5
Dificultades encontradas (problemas y decisiones tomadas para solucionarlo) ...	18

1. Introducción

Twitch se ha consolidado como la plataforma de streaming en vivo más relevante en el ámbito de los videojuegos y los eSports. Desde su lanzamiento en 2011, su crecimiento ha sido exponencial, alcanzando audiencias globales y generando ingresos millonarios. La plataforma permite a los creadores de contenido interactuar en tiempo real con sus comunidades, mientras que los espectadores pueden consumir contenido en vivo de sus streamers favoritos.

Con el auge de Twitch, la cantidad de datos generados ha crecido de manera significativa, lo que presenta tanto oportunidades como desafíos para marcas, creadores de contenido y analistas de tendencias. Identificar los canales con mayor impacto dentro de las diferentes comunidades de la plataforma no es una tarea sencilla y requiere herramientas especializadas para procesar y analizar estos datos de manera efectiva.

En respuesta a esta necesidad, hemos desarrollado **Twitch Analytics**, una API diseñada para transformar estos datos en información útil. Tras el éxito de la primera versión, hemos decidido ampliar su funcionalidad con una nueva versión que introduce mejoras significativas. Entre ellas, se destaca la incorporación de un sistema de autenticación mejorado, que permitirá a las empresas registrarse como usuarios premium y acceder a características avanzadas. Además, se ha implementado un nuevo endpoint, denominado Top of the Tops, que ofrece información sobre los tres juegos más populares en Twitch y sus vídeos más vistos.

Estas mejoras buscan proporcionar una herramienta más robusta y eficiente para los usuarios que necesitan analizar tendencias en Twitch, optimizar estrategias de marketing y maximizar el alcance de sus contenidos en la plataforma. En este documento, se detalla el desarrollo de estas nuevas funcionalidades, así como las decisiones técnicas y estratégicas adoptadas para su implementación.

2. Objetivos

El objetivo principal de la segunda fase del proyecto Twitch Analytics es ampliar las funcionalidades del sistema para mejorar su utilidad y seguridad, especialmente de cara a su potencial uso por empresas interesadas en analizar el impacto de los streamers en diversas comunidades de Twitch. Para ello, se implementarán mejoras en la API que permitirán:

1. **Implementar un sistema de autenticación seguro:** Se introducirá un mecanismo basado en API Keys y tokens temporales para controlar el acceso a la API, garantizando que solo los usuarios registrados puedan utilizar sus funcionalidades.
2. **Registrar usuarios premium:** Se añadirá un nuevo endpoint que permitirá a los usuarios registrarse proporcionando su correo electrónico, tras lo cual recibirán una API Key única para autenticarse en el sistema.
3. **Gestionar sesiones de usuario:** Se incluirá un mecanismo de obtención de tokens de sesión con una duración de 3 días, los cuales deberán ser utilizados en cada solicitud a la API para acceder a las funcionalidades protegidas.
4. **Desarrollar un nuevo endpoint “Top of the Tops”:** Se implementará un servicio que proporcionará información sobre los tres juegos más populares en Twitch y los 40 vídeos más vistos de cada uno de ellos, facilitando el análisis del contenido de mayor impacto en la plataforma.
5. **Optimizar el rendimiento mediante almacenamiento en caché:** Se diseñará un sistema que almacenará temporalmente los datos para reducir la carga de consultas a la API de Twitch. Los datos se actualizarán automáticamente cada 10 minutos, aunque se permitirá forzar una actualización manual mediante un parámetro opcional.
6. **Garantizar accesibilidad y documentación completa:** La API seguirá siendo pública y accesible desde cualquier dispositivo, y se proporcionará una documentación detallada sobre su uso, diseño, decisiones técnicas y resolución de problemas.

El proyecto continuará siendo desarrollado en PHP. Además de la implementación técnica, se entregará documentación sobre el proceso de desarrollo, incluyendo decisiones tomadas y servicios externos utilizados.

3. Consideraciones (probar implementación y dónde se encuentra alojada)

Para alojar nuestra aplicación, hemos optado por mantener la infraestructura utilizada en la primera fase del proyecto, conservando tanto el mismo **servidor** como el **subdominio** previamente configurado. De este modo, seguimos operando bajo la misma dirección IP, lo que facilita la continuidad del desarrollo y evita complicaciones en la configuración de acceso.

El servidor, que ya estaba configurado con **Ubuntu** y **PHP**, ha sido ampliado con la instalación y configuración de **MySQL**, una medida necesaria para la gestión de las bases de datos en esta nueva fase del proyecto. Durante la implementación de MySQL, se realizaron ajustes en los **permisos de acceso remoto**, lo que permite la administración y consulta de la base de datos desde ubicaciones externas de manera segura.

Una vez completada la configuración, se crearon las tablas necesarias para almacenar los datos que serán utilizados en las nuevas funcionalidades de la API. Esta infraestructura optimizada nos permite **mantener la escalabilidad y eficiencia del sistema**, asegurando que la API pueda manejar las solicitudes y el procesamiento de datos sin comprometer el rendimiento.

Con esta configuración, hemos garantizado que la aplicación cuente con un entorno estable y seguro, facilitando su desarrollo y futuras ampliaciones sin la necesidad de realizar migraciones o cambios drásticos en la infraestructura existente.

4. Detalles sobre implementación

config.php

1. Carga de Variables de Entorno

Para garantizar un acceso seguro a las credenciales del sistema, *config.php* primero verifica la existencia del archivo `.env`, que almacena de forma externa las claves y configuraciones necesarias para la aplicación. Si el archivo está presente, se extraen las siguientes variables:

- `CLIENT_SECRET` y `CLIENT_ID`: Credenciales necesarias para autenticar la aplicación con la API de Twitch.
- `DBNAME`, `SERVERNAME`, `USERNAME`, `PASSWORD`: Datos de conexión a la base de datos.
- `SECRET_KEY`: Clave utilizada para la generación y validación de tokens JWT.

Cada una de estas variables es definida como una constante en PHP, asegurando que su valor permanezca inmutable durante la ejecución del programa y esté disponible globalmente para cualquier script que requiera acceder a ellas.

2. Validación de Variables

Una vez cargadas las credenciales, *config.php* realiza una validación para garantizar que ninguna variable crítica esté ausente o vacía. Si alguna credencial no está definida correctamente, el script detiene la ejecución de inmediato y muestra un mensaje de error específico, indicando qué configuración es inválida o está faltante. Esto evita que la aplicación intente operar sin los permisos adecuados, previniendo errores en tiempo de ejecución o vulnerabilidades de seguridad.

3. Manejo de Errores

En caso de que el archivo `.env` no esté presente o no pueda ser leído, el sistema genera un mensaje de error en formato JSON para informar al usuario sobre la falta de configuración. Este enfoque facilita la depuración y permite identificar rápidamente problemas en la carga de las variables de entorno.

get_token_test.php

1. Carga de Configuración

Para operar correctamente, *get_token_test.php* necesita acceder a las credenciales almacenadas en *config.php*, por lo que lo importa al inicio del script. Además, define la ubicación del archivo *token.json*, donde se almacena temporalmente el token de acceso para reutilizarlo en futuras consultas sin necesidad de solicitar uno nuevo a Twitch.

2. Obtención del Token

Cuando la aplicación requiere un nuevo token, se realiza una solicitud POST a <https://id.twitch.tv/oauth2/token>, enviando las credenciales de la aplicación (*client_id* y *client_secret*) junto con el tipo de autenticación *grant_type=client_credentials*.

Si la solicitud es exitosa, la API de Twitch responde con un *access_token* y un tiempo de expiración (*expires_in*), indicando cuántos segundos será válido el token antes de requerir una renovación.

3. Gestión del Token

Para evitar múltiples solicitudes innecesarias a la API de Twitch, *get_token_test.php* implementa una lógica de almacenamiento y validación:

- Si el archivo *token.json* ya existe y contiene un token válido, el sistema lo reutiliza sin solicitar uno nuevo.
- Si el token ha caducado o el archivo *token.json* no está disponible, se genera un nuevo token, que es almacenado junto con su tiempo de expiración en *token.json*.

Este mecanismo optimiza el uso del servicio, reduciendo la cantidad de peticiones a la API de Twitch y evitando bloqueos o limitaciones en las solicitudes.

constructor_bbdd.php e init.sql

constructor_bbdd.php - Conexión y Ejecución del Script SQL

Este archivo se encarga de establecer una conexión con la base de datos y ejecutar el script *init.sql*, que define la estructura de las tablas.

Su funcionamiento sigue estos pasos:

1. Conexión a la Base de Datos: Se utiliza *mysqli* para conectarse al servidor de base de datos con las credenciales definidas en *config.php*.
2. Ejecución del Script SQL: Si la conexión es exitosa, se carga y ejecuta *init.sql*, asegurando que todas las tablas sean creadas correctamente.
3. Manejo de Errores: En caso de fallo en la conexión o ejecución del script, se captura el error y se muestra un mensaje detallado.

Este proceso automatiza la creación de la base de datos, evitando la necesidad de configuraciones manuales y reduciendo la posibilidad de errores en la estructura de datos.

init.sql - Creación de la Estructura de la Base de Datos

El archivo *init.sql* define las tablas y relaciones necesarias para el almacenamiento de datos. Su implementación incluye:

- Tabla **USUARIOS**: Almacena los correos electrónicos de los usuarios registrados.
- Tabla **API_KEY**: Contiene las API Keys asignadas a cada usuario, permitiendo la autenticación en la API.
- Tabla **SESSION_TOKENS**: Gestiona los tokens de sesión, asegurando un control de accesos mediante autenticación JWT.
- Definición de Relaciones: **API_KEY** y **SESSION_TOKENS** incluyen claves foráneas (**USUARIO_ID**) que establecen una relación con la tabla **USUARIOS**, asegurando integridad referencial.

Nuevo Sistema de Autenticación - 1. Registro de Usuarios

register.php

1. Validación del Correo Electrónico

Antes de registrar a un usuario, el sistema verifica la validez del correo electrónico proporcionado mediante una expresión regular, asegurando que cumpla con el formato estándar de correos electrónicos.

Además, se comprueba que el dominio del correo coincida con el dominio del servidor, evitando registros con direcciones fraudulentas o externas no autorizadas. Si el correo no es válido o no coincide con el dominio esperado, el sistema devuelve un error y bloquea el registro.

2. Gestión de Usuarios en la Base de Datos

Una vez validado el correo, el sistema revisa si el usuario ya existe en la base de datos:

- Si el usuario ya está registrado, se actualiza su API Key con una nueva clave generada.
- Si el usuario no existe, se crea un nuevo registro en la base de datos y se le asigna una API Key.

Esta gestión garantiza que cada usuario tenga una clave única y actualizada para autenticarse en la API.

3. Generación de API Keys

Para garantizar la seguridad del sistema, *register.php* genera claves seguras utilizando `random_bytes(64)`, lo que asegura una API Key altamente aleatoria y difícil de predecir.

La clave generada se almacena en la tabla `API_KEY`, asociada al usuario correspondiente, para futuras validaciones en el proceso de autenticación.

Nuevo Sistema de Autenticación - 2. Obtención de Token de Sesión

token2.php

1. Validación del Email y la API Key

Antes de generar un token JWT, el sistema verifica que el usuario haya proporcionado un email válido y una API Key existente. Para ello:

- Se comprueba que el formato del email sea correcto utilizando una expresión regular.
- Se consulta la base de datos para verificar que la API Key proporcionada corresponde al email registrado.
- Si la API Key es incorrecta o no existe en la base de datos, se rechaza la solicitud con un error 401 Unauthorized.

Este proceso garantiza que solo usuarios autenticados puedan obtener un token, evitando accesos no autorizados.

2. Generación del Token JWT

Una vez validado el usuario, el sistema genera un JWT (JSON Web Token) utilizando la biblioteca firebase/php-jwt.

El token contiene la siguiente información (payload):

- Email del usuario → Identifica al usuario autenticado.
- Tiempo de expiración → Define que el token será válido por 3 días.

El token se firma utilizando una clave secreta (SECRET_KEY), asegurando que solo el servidor pueda validar su autenticidad y prevenir manipulaciones externas.

3. Envío del Token al Cliente

Si la autenticación es exitosa y el token se genera correctamente, se devuelve en formato JSON al usuario, permitiéndole utilizarlo en futuras solicitudes a la API.

El usuario deberá incluir este token en la cabecera de autorización (Authorization: Bearer <token>) para acceder a los endpoints protegidos.

En caso de que ocurra un error durante el proceso, el sistema devuelve una respuesta adecuada con el código de estado HTTP correspondiente (400 Bad Request, 401 Unauthorized, 500 Internal Server Error), facilitando la depuración y el manejo de errores.

Nuevo Sistema de Autenticación - 3. Uso de la Autenticación en el API

validaToken.php

1. Extracción del Token

Cuando un usuario realiza una solicitud a un endpoint protegido, *validaToken.php* extrae el token JWT desde la cabecera HTTP Authorization.

Si el token no está presente, se devuelve un error 401 Unauthorized, impidiendo el acceso.

2. Verificación de Integridad

Una vez obtenido el token, el sistema lo decodifica y valida utilizando la clave secreta (SECRET_KEY). Durante este proceso:

- Se verifica que el token no haya sido alterado o manipulado.
- Se revisa su fecha de expiración para determinar si aún es válido.
- Si el token ha caducado o es inválido, la solicitud es rechazada con un error 401 Unauthorized.

Este mecanismo impide que usuarios malintencionados accedan a la API con tokens expirados o generados de manera fraudulenta.

3. Autorización del Usuario

Si el token es válido y sigue vigente, se permite el acceso al usuario y se le concede autorización para utilizar el servicio solicitado.

En caso contrario, se deniega la solicitud y se notifica al cliente con un mensaje de error, reforzando la seguridad del sistema.

CASO DE USO 1: CONSULTAR INFORMACIÓN DE UN STREAMER DE TWITCH

`info_streamer.php`

1. Validación del Token de Autenticación

El primer paso en *info_streamer.php* es garantizar que el usuario que realiza la solicitud esté autenticado mediante un token JWT (JSON Web Token). Para ello:

- Se extrae el token desde la cabecera Authorization de la solicitud.
- Se llama a la función `validarToken()`, que decodifica y valida el token usando la `SECRET_KEY`.
- Si el token es válido, la solicitud continúa su flujo normal.
- Si el token es inválido o ha expirado, el sistema responde inmediatamente con un error 401 Unauthorized, evitando accesos no autorizados a la API.

Este paso no solo protege los endpoints sensibles de la API, sino que también asegura que solo los usuarios legítimos puedan acceder a la información detallada de los streamers.

2. Verificación del ID del Streamer

Después de la validación del token, el sistema verifica que la solicitud incluya un ID de streamer válido. Para ello:

- Se comprueba que el parámetro `id` esté presente en la URL de la solicitud (`/analytics/user?id=<streamer_id>`).
- Se valida el formato del ID, asegurándose de que no contenga caracteres no permitidos o esté vacío.
- Si el ID es inválido o no se ha especificado, la API devuelve un error 400 Bad Request, indicando que la solicitud no cumple con los requisitos mínimos.

Esta verificación permite que solo se realicen consultas estructuradas y válidas, reduciendo la carga en la API de Twitch y evitando solicitudes innecesarias o erróneas.

3. Consulta a la API de Twitch

Con un token válido y un ID de streamer correcto, *info_streamer.php* procede a realizar la consulta a la API de Twitch mediante la siguiente URL:

`https://api.twitch.tv/helix/users?id=<streamer_id>`

La respuesta de la API de Twitch incluye datos clave sobre el streamer, tales como:

- ID y nombre de usuario (`login`, `display_name`).
- Tipo de cuenta (`broadcaster_type`), indicando si es afiliado, socio o usuario estándar.
- Descripción del streamer, proporcionando contexto adicional sobre su contenido.
- Imágenes de perfil (`profile_image_url`) y banner (`offline_image_url`).
- Número total de visualizaciones (`view_count`), reflejando la popularidad del streamer.
- Fecha de creación de la cuenta (`created_at`), útil para análisis históricos.

En caso de que el ID proporcionado no corresponda a ningún streamer existente, la API devuelve un error 404 Not Found, indicando que no se encontraron resultados para la consulta realizada.

4. Devolución de Datos en Formato JSON

Una vez obtenidos los datos del streamer, la API organiza la información en formato JSON, asegurando que:

- Los datos sean estructurados y legibles, facilitando su integración con otras aplicaciones o servicios.
- Se manejen adecuadamente posibles errores en la comunicación con la API de Twitch.
- En caso de fallos, se devuelva un mensaje de error coherente, con el código de estado HTTP adecuado (500 Internal Server Error).

El formato JSON también permite que la respuesta sea fácilmente consumida por clientes frontend, sistemas de análisis de datos o herramientas de monitoreo.

CASO DE USO 2: CONSULTAR STREAMS EN VIVO

`info_live_streams.php`

1. Validación del Token de Autenticación

El primer paso en *info_live_streams.php* es asegurarse de que la solicitud provenga de un usuario autenticado. Para ello, el sistema lleva a cabo una validación exhaustiva del token JWT:

- Se extrae el token desde la cabecera Authorization de la solicitud HTTP.
- Se llama a la función `validarToken()`, que verifica:
 - La integridad del token utilizando la clave secreta (`SECRET_KEY`).
 - Que el token no haya caducado, comparando su fecha de expiración con la fecha actual.
- Si el token es válido, la solicitud continúa su flujo normal.
- Si el token es inválido o ha expirado, la API responde con un error 401 Unauthorized, impidiendo el acceso no autorizado a la información de los streams en vivo.

Asegura que solo usuarios legítimos puedan interactuar con la API, protegiendo los endpoints sensibles y evitando accesos no autorizados que podrían comprometer la seguridad y estabilidad del sistema.

2. Consulta a la API de Twitch

Con un token validado, el sistema procede a consultar la API de Twitch para obtener la lista de streams en vivo:

- Se envía una solicitud a `https://api.twitch.tv/helix/streams` utilizando el token de acceso de Twitch previamente obtenido mediante *get_token_test.php*.
- La consulta se realiza con las cabeceras adecuadas:
 - Authorization: Bearer <access_token> para autenticar la solicitud.
 - Client-ID para identificar la aplicación ante la API de Twitch.

Datos Clave Obtenidos:

La respuesta de Twitch proporciona una lista de streams activos, incluyendo información como:

- Título de la transmisión: Permite identificar el contenido de cada stream.
- Nombre del usuario: Muestra quién está realizando la transmisión.
- Cantidad de espectadores en tiempo real: Indicador de la popularidad del stream.

El sistema procesa esta información para estructurarla y enriquecerla, asegurando que los datos sean claros y precisos antes de ser devueltos al cliente.

3. Envío de la Respuesta

Una vez obtenidos y procesados los datos de Twitch, la API devuelve la información en formato JSON, garantizando una estructura clara y estandarizada:

- Formato JSON: Facilita la integración con otros sistemas o aplicaciones frontend, permitiendo un fácil consumo de los datos.
- Manejo de Errores:
 - Si no hay streams activos, la API responde con un error 404 Not Found.
 - Si ocurre un error en la comunicación con Twitch, se devuelve un 500 Internal Server Error, indicando un problema del lado del servidor.

El sistema también implementa mensajes de error personalizados, proporcionando detalles útiles para la depuración y la gestión de excepciones.

CASO DE USO 3: CONSULTAR “TOP STREAMS ENRIQUECIDOS”

`get_top_streams.php`

1. Validación del Token de Autenticación

El primer paso antes de consultar la API de Twitch es validar el token JWT del usuario que realiza la solicitud. Esta autenticación previa garantiza que solo usuarios autorizados puedan acceder a la información de los streams más populares.

- Se obtiene el token desde la cabecera Authorization de la solicitud HTTP.
- Se llama a `validarToken()`, que decodifica y verifica el token utilizando la clave secreta (`SECRET_KEY`).
- Si el token ha caducado o es inválido, se devuelve un error 401 Unauthorized, impidiendo el acceso no autorizado.

Este proceso asegura que la API mantenga un alto nivel de seguridad, protegiendo los endpoints sensibles y garantizando un acceso controlado a los datos de Twitch.

2. Consulta a la API de Twitch

Una vez validado el usuario, el sistema realiza una solicitud directa a la API de Twitch para obtener la lista de streams en vivo.

- URL de la Solicitud: <https://api.twitch.tv/helix/streams>
- Autenticación en la Solicitud:
 - Se envían las cabeceras necesarias:

Authorization: Bearer <access_token>

Client-ID: <client_id>
 - El token de acceso de Twitch se obtiene previamente mediante el módulo `get_token_test.php`, asegurando que esté actualizado y vigente.
- Filtrado de Resultados:
 - Una vez recibida la respuesta, el sistema filtra los streams con base en el número de espectadores (`viewer_count`).
 - Se priorizan aquellos con mayor audiencia, proporcionando así una lista de streams ordenada de mayor a menor popularidad.

Datos Clave Obtenidos:

- ID del stream y del usuario.
- Nombre del usuario (`user_name`).
- Título de la transmisión.
- Número de espectadores en tiempo real (`viewer_count`).

Este proceso asegura que la API siempre proporcione información actualizada y relevante, directamente desde la fuente oficial de Twitch.

3. Enriquecimiento de Datos

Para ofrecer una experiencia más completa, el sistema enriquece los datos obtenidos con información adicional sobre los streamers.

- Segunda Consulta a la API de Twitch:
 - Se utiliza la API de usuarios de Twitch para obtener detalles adicionales de cada streamer, como:

`https://api.twitch.tv/helix/users?id=<user_id>`
 - Esta consulta devuelve datos adicionales que se integran con la información original de los streams.
- Datos Enriquecidos:
 - Nombre de usuario (`display_name`): Proporciona una identificación clara del streamer.
 - Imagen de perfil (`profile_image_url`): Añade un componente visual a la información del streamer.

Este enriquecimiento mejora la calidad de los datos devueltos, permitiendo a los consumidores de la API acceder a más contexto e información visual.

4. Ordenación y Respuesta en Formato JSON

Después de enriquecer los datos, la API organiza y devuelve la información en formato JSON estructurado, asegurando que sea fácil de interpretar y consumir por aplicaciones externas.

- Ordenación de los Streams:
 - Se ordena la lista de streams de mayor a menor número de espectadores, colocando los streams más populares al inicio de la respuesta.
- Estructuración en JSON:
 - Se genera un JSON estandarizado, ideal para integraciones con frontends, sistemas de análisis o servicios externos.
 - Se incluye un manejo adecuado de errores, proporcionando mensajes claros y códigos de estado HTTP adecuados:
 - 404 Not Found si no se encuentran streams activos.
 - 500 Internal Server Error si ocurre un fallo en la comunicación con Twitch.

router.php

1. Definición de Endpoints Protegidos

Para reforzar la seguridad, *router.php* define una lista de rutas protegidas que requieren autenticación previa. Estas incluyen endpoints que manejan información sensible, como:

- */analytics/user* → Consulta de datos de un streamer.
- */analytics/streams* → Consulta de streams en vivo.
- */analytics/streams/enriched* → Obtención de los streams más populares con datos enriquecidos.

Cualquier solicitud dirigida a estas rutas deberá pasar por un proceso de autenticación antes de ser procesada.

2. Verificación del Token

Antes de permitir el acceso a un endpoint protegido, *router.php* ejecuta el método `validarToken()`, el cual comprueba si el usuario ha proporcionado un token JWT válido en la cabecera de la solicitud.

Si el token es inválido o ha expirado, la solicitud es rechazada con un error 401 Unauthorized, impidiendo el acceso no autorizado a la API.

3. Redirección de Solicitudes

Una vez validado el token (si es necesario), *router.php* determina a qué archivo debe ser redirigida la solicitud en función de la URL recibida.

Por ejemplo:

- Si la solicitud es a */analytics/user*, se carga *info_streamer.php*.
- Si es a */analytics/streams*, se redirige a *info_live_streams.php*.
- Si es a */analytics/streams/enriched*, se redirige a *get_top_streams.php*.
- Si se solicita */register*, se redirige a *register.php* para el registro de usuarios.
- Si se solicita */token*, se redirige a *token2.php* para la obtención del token de sesión.
- Si la URL no coincide con ningún endpoint válido, se devuelve un error 404 Not Found.

Esta estructura modular permite mantener el código organizado y facilita la ampliación de la API, ya que agregar un nuevo endpoint solo requiere incluirlo en *router.php*.

5. Dificultades encontradas (problemas y decisiones tomadas para solucionarlo)

Problema 1: Configuración del Firewall para Acceso Remoto a la Base de Datos

Tras instalar la base de datos en el servidor, descubrimos que no era posible acceder a ella de forma remota. Las solicitudes externas al puerto 3306, utilizado por MySQL, eran bloqueadas sistemáticamente. Esto se debía a la configuración del firewall del servidor, que restringía todas las conexiones entrantes en ese puerto, impidiendo el acceso desde ubicaciones externas. Esta limitación comprometía la capacidad de los desarrolladores para realizar pruebas y acceder a los datos de manera eficiente.

Análisis del Problema y Alternativas Consideradas

1. Restricción de Acceso por IP Específica:

Inicialmente, intentamos solucionar el problema configurando el firewall para permitir conexiones solo desde una IP específica. Este enfoque, aunque seguro, demostró ser poco práctico ya que las IP públicas de los desarrolladores eran dinámicas, cambiando con frecuencia según la ubicación o la red utilizada. Esta variabilidad generó constantes problemas de acceso, retrasando el flujo de trabajo y afectando la productividad del equipo.

2. Apertura del Puerto a Todas las IP (0.0.0.0/0):

Dado que la solución inicial no era viable, decidimos permitir conexiones entrantes al puerto 3306 desde cualquier dirección IP (0.0.0.0/0). Esta decisión garantizó la accesibilidad global a la base de datos, permitiendo que cualquier miembro del equipo pudiera conectarse sin importar su ubicación. Sin embargo, reconocimos que esta configuración era menos segura, por lo que tomamos medidas adicionales para mitigar los riesgos de seguridad asociados.

Acciones Implementadas para Resolver el Problema

1. Configuración del Firewall

- Creación de una regla específica: Configuramos el firewall del servidor para permitir conexiones entrantes en el puerto 3306, asegurando la accesibilidad remota.
- Apertura a todas las IP: Establecimos la regla de acceso a 0.0.0.0/0, eliminando las restricciones basadas en IP específica. Esta configuración proporcionó la flexibilidad necesaria para el acceso remoto, aunque introdujo la necesidad de implementar medidas de seguridad adicionales.

2. Ajuste de Permisos en MySQL

Además de modificar el firewall, realizamos cambios en la configuración de MySQL para permitir conexiones remotas de forma segura:

- Asignación de Privilegios al Usuario de MySQL: Ajustamos los permisos del usuario, permitiendo el acceso desde cualquier IP ('usuario'@'%') y asegurándonos de que solo tuviera los privilegios necesarios (SELECT, INSERT, UPDATE).
- Modificación del Archivo de Configuración (my.cnf): Establecimos la propiedad bind-address en 0.0.0.0, lo que permitió a MySQL escuchar conexiones desde todas las interfaces de red.
- Reinicio del Servicio: Aplicamos las nuevas configuraciones mediante el reinicio del servicio de MySQL, garantizando que las reglas de acceso remoto entraran en vigor.

Hemos tenido problemas con el servidor a última hora. Ambas máquinas están en RUNNING, pero las reglas de Firewall al crear otras entran en conflicto entre sí.